

Ex. No.: 12
Date: 11/04/25

File Organization Technique- Single and Two level directory

AIM:

To implement File Organization Structures in C are

- a. Single Level Directory
- b. Two-Level Directory
- c. Hierarchical Directory Structure
- d. Directed Acyclic Graph Structure

a. Single Level

Directory

ALGORITHM

1. Start
2. Declare the number, names and size of the directories and file names.
3. Get the values for the declared variables.
4. Display the files that are available in the directories.
5. Stop.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

struct File {
    char name[20];
};

int main() {
    struct File files[10];
    int count = 0;
    int choice;
    char fname[20];

    do {
```

```
printf("\n--- Single Level Directory ---\n");
printf("1. Create File\n2. Delete File\n3. Search File\n4. Display Files\n5. Exit\nEnter choice: ");
scanf("%d", &choice);
```

```
switch (choice) {
    case 1:
        printf("Enter file name to create: ");
        scanf("%s", fname);
        int exists = 0;
        for (int i = 0; i < count; i++) {
            if (strcmp(files[i].name, fname) == 0) {
                exists = 1;
                break;
            }
        }
        if (exists) {
            printf("File already exists!\n");
        } else {
            strcpy(files[count++].name, fname);
            printf("File created.\n");
        }
        break;
```

```
    case 2:
        printf("Enter file name to delete: ");
        scanf("%s", fname);
        int found = 0;
        for (int i = 0; i < count; i++) {
            if (strcmp(files[i].name, fname) == 0) {
                for (int j = i; j < count - 1; j++) {
                    files[j] = files[j + 1];
                }
                count--;
                found = 1;
                printf("File deleted.\n");
                break;
            }
        }
        if (!found)
            printf("File not found!\n");
        break;
```

```
    case 3:
        printf("Enter file name to search: ");
        scanf("%s", fname);
        found = 0;
        for (int i = 0; i < count; i++) {
            if (strcmp(files[i].name, fname) == 0) {
                found = 1;
                printf("File found.\n");
                break;
            }
        }
```

```

    }
    if (!found)
        printf("File not found!\n");
    break;

case 4:
    printf("Files in directory:\n");
    for (int i = 0; i < count; i++) {
        printf("%s\n", files[i].name);
    }
    break;
}
} while (choice != 5);

return 0;
}

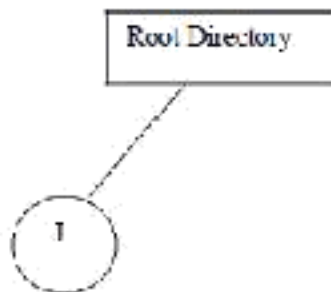
```

OUTPUT:

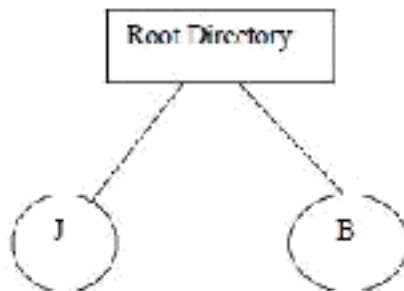
Enter the Number of files

2

Enter the file1 J



Enter the file2 B



b. Two-level directory Structure

ALGORITHM:

1. Start
2. Declare the number, names and size of the directories and subdirectories and file names.
3. Get the values for the declared variables.
4. Display the files that are available in the directories and subdirectories.
5. Stop.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

struct File {
    char name[20];
};

struct Directory {
    char user[20];
    struct File files[10];
    int file_count;
};

int main() {
    struct Directory dirs[5];
    int dir_count = 0;
    int choice;
    char uname[20], fname[20];

    do {
        printf("\n--- Two-Level Directory ---\n");
        printf("1. Create User Directory\n2. Create File\n3. Display User Files\n4. Exit\nEnter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter user name: ");
                scanf("%s", uname);
                int exists = 0;
                for (int i = 0; i < dir_count; i++) {
```

```

        if (strcmp(dirs[i].user, uname) == 0) {
            exists = 1;
            break;
        }
    }
    if (exists)
        printf("User already exists!\n");
    else {
        strcpy(dirs[dir_count].user, uname);
        dirs[dir_count].file_count = 0;
        dir_count++;
        printf("User directory created.\n");
    }
    break;

```

case 2:

```

    printf("Enter user name: ");
    scanf("%s", uname);
    int userIndex = -1;
    for (int i = 0; i < dir_count; i++) {
        if (strcmp(dirs[i].user, uname) == 0) {
            userIndex = i;
            break;
        }
    }
    if (userIndex == -1) {
        printf("User not found!\n");
        break;
    }
    printf("Enter file name to create: ");
    scanf("%s", fname);
    int fileExists = 0;
    for (int j = 0; j < dirs[userIndex].file_count; j++) {
        if (strcmp(dirs[userIndex].files[j].name, fname) == 0) {
            fileExists = 1;
            break;
        }
    }
    if (fileExists)
        printf("File already exists!\n");
    else {
        strcpy(dirs[userIndex].files[dirs[userIndex].file_count++].name, fname);
        printf("File created under user %s.\n", uname);
    }
    break;

```

case 3:

```

    printf("Enter user name: ");
    scanf("%s", uname);
    userIndex = -1;
    for (int i = 0; i < dir_count; i++) {

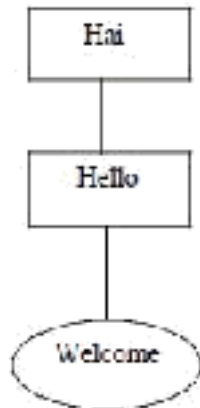
```

```
        if (strcmp(dirs[i].user, uname) == 0) {
            userIndex = i;
            break;
        }
    }
    if (userIndex == -1) {
        printf("User not found!\n");
        break;
    }
    printf("Files for user %s:\n", uname);
    for (int j = 0; j < dirs[userIndex].file_count; j++) {
        printf("%s\n", dirs[userIndex].files[j].name);
    }
    break;
}
} while (choice != 4);

return 0;
}
```

Sample Output:

Enter the name of dir/file(under null): Hai
How many users(for Hai):1
Enter name of dir/file(under Hai):Hello
How many files(for Hello):1
Enter name of dir/file(under Hello):welcome

**Result:**

Thus, the program is compiled and executed successfully.